

Name - Chaitanya Kelkar

Roll no - 15

Experiment No 7

Aim

To implement the Ganache blockchain platform for local Ethereum development, connect it with MetaMask and Remix IDE, and deploy and interact with a Solidity smart contract in a fully local test environment.

Theory

Ganache

Ganache is a personal, local Ethereum blockchain used for development and testing, developed as part of the Truffle Suite. Unlike public testnets such as Sepolia, which run on a distributed network of real nodes, Ganache runs entirely on the developer's own machine and exists only in memory — meaning it resets completely every time the application is restarted. It automatically pre-funds a set of test accounts with 100 ETH each, mines transactions instantly rather than waiting for network consensus, and provides a graphical interface to inspect accounts, blocks, transactions, and event logs directly, without needing an external block explorer.

Local Blockchain vs. Public Testnet

A local blockchain like Ganache mirrors the core mechanics of Ethereum — accounts, transactions, gas, blocks — but is isolated from the outside world. This makes it ideal for rapid, cost-free iteration during early development, before a contract is mature enough to be tested on a shared public network like Sepolia. The trade-off is that nothing deployed on Ganache is independently verifiable by anyone else, since the chain is private to the developer's own computer.

Custom RPC Networks in MetaMask

MetaMask can connect to any Ethereum-compatible blockchain, not just well-known networks, by adding a Custom RPC Network. This requires specifying the RPC URL (the address where the blockchain accepts requests — <http://127.0.0.1:7545> for Ganache), a Chain ID (a unique identifier for that specific network, 1337 for Ganache by default), and a currency symbol. Once added, MetaMask treats it exactly like Sepolia or Mainnet — the same "Injected Provider" model used in Remix applies without any special integration effort.

Importing Externally-Owned Accounts

Ganache generates its own set of accounts and private keys at startup, separate from any wallet a developer may already have. To use these pre-funded accounts, their private keys must be manually imported into MetaMask as new accounts. Once imported, MetaMask can sign

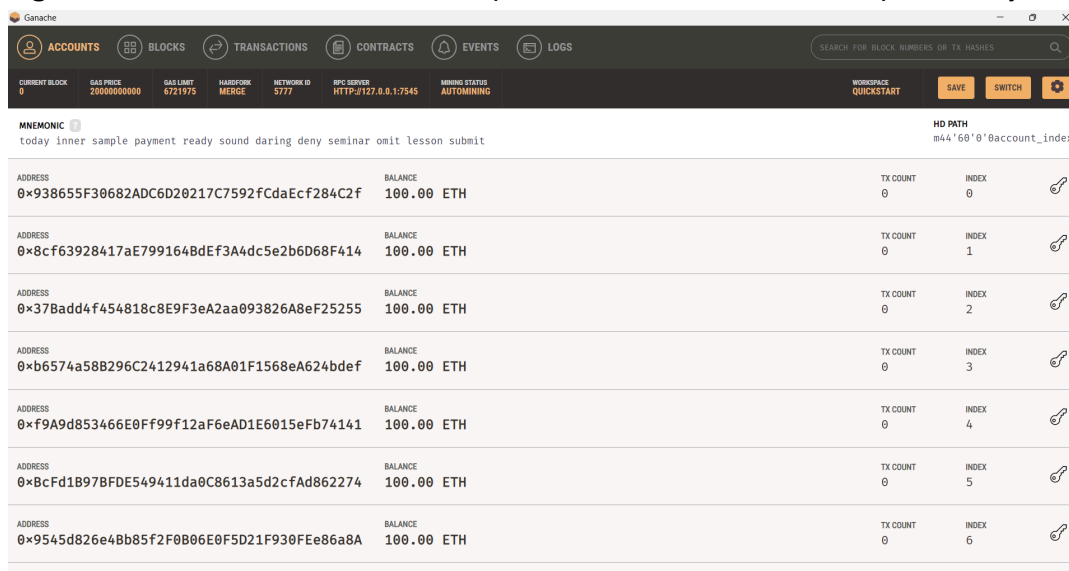
transactions on behalf of that account exactly as it would for any other wallet, allowing Remix to deploy and interact with contracts using Ganache's funded test accounts.

EVM Version Compatibility

Solidity compilers target a specific EVM (Ethereum Virtual Machine) version, which determines which low-level opcodes the compiled bytecode may contain. Newer compiler defaults may generate opcodes (such as **PUSH0**, introduced in the Shanghai upgrade) that older blockchain simulators like Ganache do not recognize, causing deployment to fail with an "invalid opcode" error. This is resolved by explicitly setting an older, compatible EVM version (e.g., "London") in the compiler settings before compiling, ensuring the generated bytecode only uses opcodes the target chain supports.

Outputs

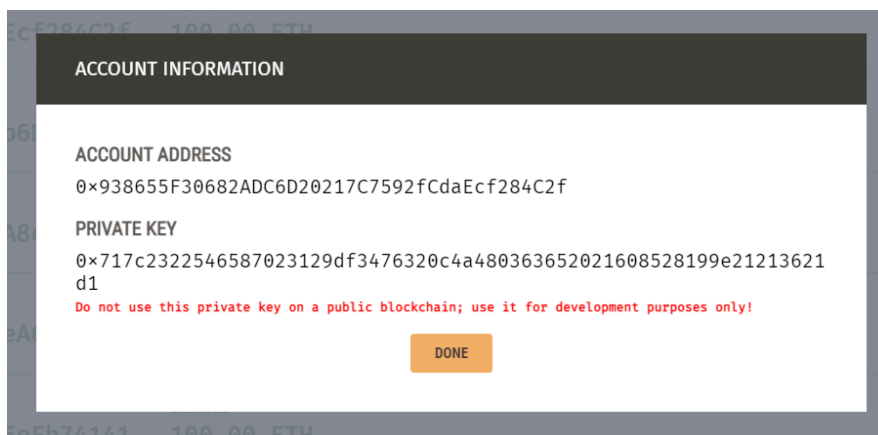
Fig 1: Ganache local blockchain with pre-funded test accounts and private key



The screenshot shows the Ganache desktop application. At the top, there's a navigation bar with tabs for ACCOUNTS, BLOCKS, TRANSACTIONS, CONTRACTS, EVENTS, and LOGS. Below this is a status bar with various metrics like CURRENT BLOCK, GAS PRICE, GAS LIMIT, HARDFORK, NETWORK ID, RPC SERVER, and MINING STATUS. The main area displays a list of accounts with columns for ADDRESS, BALANCE, TX COUNT, and INDEX. Each account has a balance of 100.00 ETH and a TX COUNT of 0. The accounts are indexed from 0 to 6. A mnemonic phrase is visible at the top left of the account list, and an HD PATH is shown at the top right.

ADDRESS	BALANCE	TX COUNT	INDEX
0x938655F30682ADC6D20217C7592fCdaEcF284C2f	100.00 ETH	0	0
0x8cf63928417aE799164BdEf3A4dc5e2b6D68F414	100.00 ETH	0	1
0x37Badd4f454818c8E9F3eA2aa093826A8eF25255	100.00 ETH	0	2
0xb6574a58B296C2412941a68A01F1568eA624bdef	100.00 ETH	0	3
0xf9A9d853466E0Ff99f12aF6eAD1E6015eFb74141	100.00 ETH	0	4
0xBcFd1B97BFDE549411da0C8613a5d2cfAd862274	100.00 ETH	0	5
0x9545d826e4Bb85f2F0B06E0F5D21F930FEe86a8A	100.00 ETH	0	6

Fig 2: Adding Ganache as a custom RPC network and importing a Ganache test account into MetaMask



The screenshot shows the MetaMask 'ACCOUNT INFORMATION' dialog box. It displays the ACCOUNT ADDRESS and PRIVATE KEY for a specific account. A warning message at the bottom states: 'Do not use this private key on a public blockchain; use it for development purposes only!'. A 'DONE' button is located at the bottom right of the dialog.

ACCOUNT INFORMATION

ACCOUNT ADDRESS
0x938655F30682ADC6D20217C7592fCdaEcF284C2f

PRIVATE KEY
0x717c2322546587023129df3476320c4a480363652021608528199e21213621d1

Do not use this private key on a public blockchain; use it for development purposes only!

DONE

Add network

Add from ChainList

Network name

Ganache Local

According to our records, the network name may not correctly match this chain ID.
Suggested name: [Localhost 8545](#)

Default RPC URL

127.0.0.1:7545

Chain ID

1337

Currency symbol

ETH

Block explorer URL

Add a URL

Save

Imported accounts

Imported Account 1

\$0.00

0x93865...84C2f

Fig 3: Connecting Remix to the Ganache Local network via MetaMask, showing imported account balance

Environment

Reset

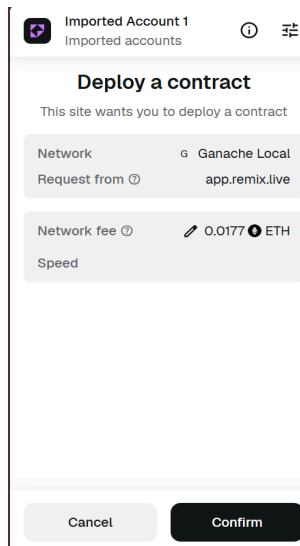
WalletConnect

Account 17

0x938...84c2f

100.000 ETH

Fig 4: Confirming and completing smart contract deployment on Ganache Local



status	1 Transaction mined and execution completed
transaction hash	0xadfbfd061621d6d626ac9503784f9b475fa6ffe4da01e30ee0f2e5add1b2fc82
block hash	0x23c39b10b5130784590a55891d4253bf094930cab33aa89cc2f7250a89f7efa8
block number	4
contract address	0x9627e81b1723e634748b482a38d5a77b06973007
from	0x938655f30682ADC6D20217C7592fCdaEcf284C2f
to	StudentMarks.(constructor)
transaction cost	883600 gas
decoded input	{}
decoded output	-

Fig 5: Calling the addStudent() function on the Ganache-deployed contract

status	1 Transaction mined and execution completed
transaction hash	0x30f9718993204ddc5fed247ad0811ad5bf8ac9cf2b035a063d2c7f746a30bc4e
block hash	0x801e8d22a3c4c93b1a6f522694fda4a0511909e26621bfa3539787e4baf1e556
block number	5
from	0x938655f30682ADC6D20217C7592fCdaEcf284C2f
to	StudentMarks.addStudent(string,uint256) 0x9627e81b1723e634748b482a38d5a77b06973007
transaction cost	139414 gas
decoded input	{ "string _name": "Chaitanya", "uint256 _marks": "100" }

